```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix

# -----------------------------------
# Load Dataset
# -----------------------------------
df = pd.read_csv("heart.csv")

print("Dataset Shape:", df.shape)

print("\nFirst 5 Rows:")
print(df.head())

print("\nMissing Values:")
print(df.isnull().sum())

# -----------------------------------
# Features and Target
# -----------------------------------
X = df.drop("target", axis=1)
y = df["target"]

# -----------------------------------
# Train Test Split
# -----------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# ===================================
# DECISION TREE
# ===================================
dt_model = DecisionTreeClassifier(
    max_depth=4,
    random_state=42
)

dt_model.fit(X_train, y_train)

dt_pred = dt_model.predict(X_test)

dt_accuracy = accuracy_score(y_test, dt_pred)

print("\nDecision Tree Accuracy:")
print(dt_accuracy)

print("\nDecision Tree Classification Report:")
print(classification_report(y_test, dt_pred))

# -----------------------------------
# Confusion Matrix
# -----------------------------------
plt.figure(figsize=(6,4))
sns.heatmap(
    confusion_matrix(y_test, dt_pred),
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title("Decision Tree Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
plt.show()

# -----------------------------------
# Decision Tree Visualization
# -----------------------------------
plt.figure(figsize=(20,10))

plot_tree(
    dt_model,
    feature_names=X.columns,
    class_names=["No Disease", "Disease"],
    filled=True
)

plt.title("Decision Tree")
plt.show()

# ==================================
# RANDOM FOREST
# ==================================
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train, y_train)

rf_pred = rf_model.predict(X_test)

rf_accuracy = accuracy_score(y_test, rf_pred)

print("\nRandom Forest Accuracy:")
print(rf_accuracy)

print("\nRandom Forest Classification Report:")
print(classification_report(y_test, rf_pred))

# -----------------------------------
# Accuracy Comparison
# -----------------------------------
models = ["Decision Tree", "Random Forest"]
accuracies = [dt_accuracy, rf_accuracy]

plt.figure(figsize=(6,4))
plt.bar(models, accuracies)

plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
plt.show()

# ==================================
# Feature Importance
# ==================================
importance = rf_model.feature_importances_

feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": importance
})

feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)

print("\nFeature Importance:")
print(feature_importance)

plt.figure(figsize=(10,6))

sns.barplot(
    x="Importance",
    y="Feature",
    data=feature_importance
)

plt.title("Random Forest Feature Importance")
plt.show()

# ==================================
# Cross Validation
# ==================================
cv_scores = cross_val_score(
```

```
    rf_model,
    X,
    y,
    cv=5
)

print("\nCross Validation Scores:")
print(cv_scores)

print("\nAverage CV Score:")
print(cv_scores.mean())

# ==================================
# Insights
# ==================================
print("\nInsights:")
print("1. Decision Tree is easy to interpret.")
print("2. Limiting max_depth helps reduce overfitting.")
print("3. Random Forest combines many trees using bagging.")
print("4. Random Forest usually provides better accuracy.")
print("5. Feature importance shows which attributes affect heart disease prediction most.")
print("6. Cross-validation gives a more reliable performance estimate.")
```